

OpsFlow 系统架构设计

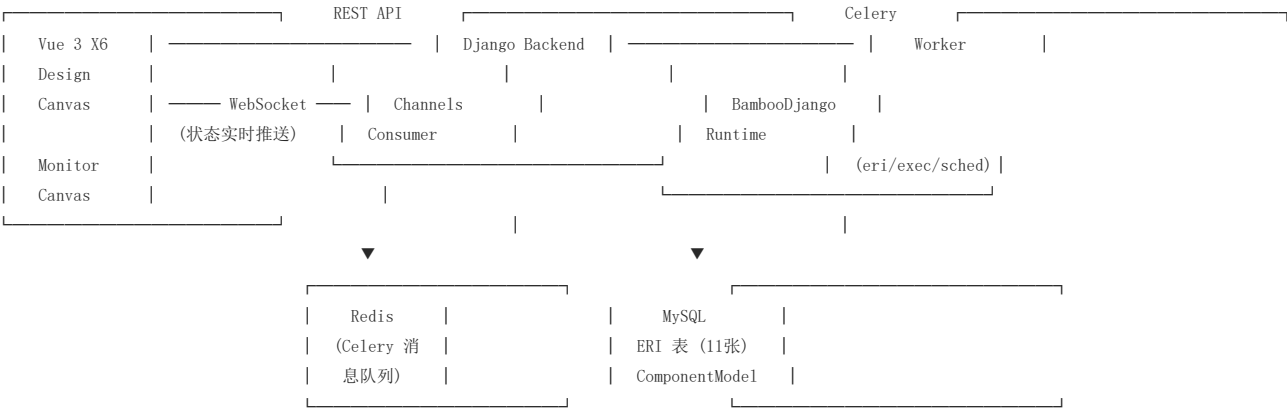
系统架构 — 分层设计 + 执行流程 + 关键决策

2026-06-03

Generated: 2026-06-03 | Source: backend/opsflow/doc/

整体架构

OpsFlow 采用**前后端分离 + 异步任务 + WebSocket 推送 + 多平台原子层**的架构。



分层设计

1. 前端展示层

- **DesignCanvas**: X6 Graph 画布，支持拖拽式编排，含 Stencil（组件面板）、Minimap（小地图）、PropertyPanel（属性面板）
- **MonitorCanvas**: 只读画布，通过 WebSocket 接收节点状态并实时着色（含 Tower 作业进度）
- **AI Chat**: 浮窗式对话界面，支持多轮交互生成/修改 Pipeline
- **DiffModal**: 对比 AI 原始生成和当前修改
- **SubmitWizardDialog**: 执行提交向导弹窗（选择方案 + 变量覆盖 + Dry Run 预览）
- **ProjectSwitcher**: 项目切换器，根据当前用户的项目成员角色切换数据范围
- **SchemeSelector/Manager**: 执行方案选择和管理界面
- **Webhook 管理**: 模板 Webhook 配置 + 投递日志查看

2. API 层

- RESTful API ([/api/opsflow/*]): 模板 CRUD、执行 CRUD、日志、知识库
- WebSocket ([ws/opsflow/execution/{id}/]): 实时推送节点状态 + Tower 作业状态
- Action 端点: [start/pause/resume/retry/skip]、[create_from_ai/refine/analyze/ai_layout]

3. 核心引擎层

- **build_bamboo_pipeline()**: 将前端 nodes/edges 自定义格式转换为 bamboo-engine 标准 Pipeline Tree
- **bamboo_validator()**: Pipeline Tree 兼容性验证 (网关配对/出入度/环检测/条件引用)
- **FlowEngine**: 流程执行引擎, 使用 BambooDjangoRuntime + api.run_pipeline() 驱动执行
- **PluginService**: 实现 bamboo-engine Service 接口, 统一路由到 BasePlugin 子类执行
- **signals/ 包**: post_set_state 信号处理器 (6 个模块), 异步追踪节点状态 → FlowExecution/NodeExecutionTrace/OpsLog
- **NodeDispatcher**: 节点操作调度器, 封装 retry/skip/force_fail + 轨迹和日志管理
- **SubprocessDispatcher**: 独立子流程调度器, 创建子 FlowExecution + 变量映射解析
- **VariableResolver**: \${key} 模板变量替换引擎, 支持 splice/split/lazy 类型
- **VariableRegistry**: bk_sops 适配变量类型 (SpliceVariable/LazyVariable 等 12 种)
- **TowerService**: Ansible Tower (AWX) REST API 封装 (触发/轮询/结果提取)
- **AnsibleTrigger**: Ansible Tower HTTP 触发器, 未配置时自动降级为模拟执行
- **Layout Engine**: Sugiyama 分层图布局引擎 (bk_sops drawing_new 适配), 纯 Python 毫秒级
- **AutoRetry**: 节点 FAILED 时自动触发重试, 支持最大次数和重试间隔
- **NodeTimeoutStrategy**: 节点超时检测 + 可配置动作 (强制失败/强制失败并跳过)
- **NodeSync**: pipeline_tree JSON TemplateNode/ExecutionNode 双向同步
- **MakoResolver**: Mako 模板引擎变量替换
- **ConflictChecker**: 多人编辑时的 pipeline_tree 冲突检测
- **TraceLogger**: 节点执行轨迹的 JSON Lines 文件日志
- **SchedulerService**: APScheduler 封装, 一次性/CRON 计划调度
- **SafetyGuard**: Pipeline 安全校验 (白名单/高危/备份/AI 幻觉防御)
- **LLMService**: DeepSeek AI 服务 (生成/修改/分析 + RAG 知识库检索)
- **APIGW**: 外部 API 网关 (API Token 认证 + 触发/状态/模板列表端点)
- **WebhookService**: Webhook 回调投递 + HMAC 签名 + 重试机制
- **AuditLogger**: 操作审计日志记录器
- **PluginDeprecation**: 插件生命周期管理 (可用/即将弃用/已弃用)

4. 插件/原子层

- **BasePlugin**: 抽象基类, 定义 execute/schedule/rollback/get_form_config/get_output_schema 契约
- **PLUGIN_REGISTRY**: 自动扫描 plugins/ 目录, 43 个原子按 group 分组, 支持多版本
- **PluginServiceAdapter**: 统一 Service 入口, 运行时从 PLUGIN_REGISTRY 查找并调用对应 BasePlugin 执行
- **PluginMeta**: 数据库持久化插件元数据 (form_schema + output_schema), 前端 RenderForm 解析

- ****schema/form_schema.py****: Pydantic 表单配置协议 (FormItem/FormGroup/ValidationRule/FormEvent)

5. 数据层

- ****MySQL (opsflow 自有表)****: FlowTemplate / TemplateVersion / FlowExecution / NodeExecutionTrace / ExecutionNode / TemplateNode / ExecutionScheme / OpsLog / SchedulePlan / OpsKnowledge / PluginMeta / OpsProject / ProjectMember / TemplateCollect / TemplateCategory / OperationRecord / AutoRetryStrategy / NodeTimeoutConfig / WebhookConfig / WebhookLog / ApiToken / ProjectEnvironmentVariable — 共 20+ 张表
- ****MySQL (bamboo-engine ERI 表)****: 11 张内置状态表 (Process/Node/ExecutionData 等) + ComponentModel
- ****Redis****: Celery 消息队列、Channels WebSocket 层

执行流程

完整 Pipeline 执行时序

用户点击“执行”



POST /api/opsflow/executions/{id}/start/



FlowEngine.start()

| 写入 execution.status = “running”
| execution.started_at = now()



execute_pipeline_task.delay(execution_id) ← Celery 异步



FlowEngine.run()

```

|
├─ build_bamboo_pipeline(template) → Pipeline Tree dict
├─ 保存 bamboo_pipeline_id 到 context
|
├─ api.run_pipeline(runtime=BambooDjangoRuntime(), pipeline=pipeline)
|
|   BambooDjangoRuntime 内部驱动执行:
|   └─ ProcessMixin → 创建 Process/Node 记录
|   └─ TaskMixin → 调度到 Celery er_execute/er_schedule 队列
|   └─ StateMixin → 状态机转换 + 发送 post_set_state 信号
|   └─ ActivityMixin → 查找 Service → AnsibleAtomService.execute()
|   |
|   |   每个 activity 执行:
|   |   1. BambooDjangoRuntime.get_service(code) → AnsibleAtomService
|   |   2. → AtomExecutorFactory → AnsibleExecutor
|   |   3. → TowerService.launch_job() POST /launch/
|   |   4. → TowerService.poll_job() 自适应轮询
|   |   5. → TowerService.extract_result() artifacts
|   |   6. 成功 → FINISHED / 失败 → FAILED
|   |
|   └─ GatewayMixin → ExclusiveGateway / ParallelGateway / ConvergeGateway
|   └─ 完成 → 根节点 FINISHED
|
└─ run_pipeline 异步返回（实际执行在 Celery 队列中）

```

节点状态变更（异步，通过信号）：



post_set_state 信号 (pipeline.eri.signals)

```

|
├─ node_id == root_id:
|   FINISHED  → execution.status = "completed"
|   FAILED    → execution.status = "failed"
|   SUSPENDED → execution.status = "paused"
|   RUNNING   → execution.status = "running"
|   _notify_completed() → WS group_send
|
├─ 子节点 FINISHED/FAILED:
|   _log_node_result() → 从 ERI Data 读取 outputs → OpsLog
|   _notify_node_status() → WS 推送节点状态

```

Tower 交互子流程

```

AnsibleExecutor.execute()
|
├─ TowerService.launch_job()
|   └─ POST /api/v2/job_templates/{id}/launch/
|   └─ extra_vars = {opsflow_atom_type, params, ...}
|   └─ 返回 job_id
|
├─ TowerService.poll_job(job_id, execution_id, node_id)
|   └─ 自适应轮询 (3s/5s/10s/30s)
|   └─ WebSocket → tower_job_update (status/progress)
|   └─ 完成/失败/超时
|
├─ TowerService.extract_result(job_id)
|   └─ GET /artifacts/ (set_stats 数据)
|   └─ GET /job_events/ (详细事件)
|   └─ GET /stdout/ (日志输出)
|   └─ 返回 {status, artifacts, events, stdout, summary}
|
├─ 返回 ExecuteResult(success, data)
|   └─ data.outputs 包含 {stdout, artifacts, summary, ...}
|       └─ flow_engine 注入 context[node_id] = {status, artifacts}

```

关键设计决策

1. 迁移到 BambooDjangoRuntime

原 FlowEngine 实现了一个自定义 Pipeline Tree 解释器，手动解析 activities/gateways/flows、用 Redis 计数器实现 ConvergeGateway、用 Celery group 实现 ParallelGateway。这套逻辑本质上是重新实现了 bamboo-pipeline 的 [BambooDjangoRuntime]。

****迁移后****: delegating to [BambooDjangoRuntime] + [api.run_pipeline()]。

旧实现	新实现
`_execute_bamboo()` 手动遍历	`api.run_pipeline(runtime, pipeline)`
`_execute_activity()` 直接调 Service	BambooDjangoRuntime.ActivityMixin
`_execute_exclusive()` 条件遍历	BambooDjangoRuntime.GatewayMixin
`_execute_parallel()` Celery group	BambooDjangoRuntime.GatewayMixin
`_execute_converge()` Redis 计数器	BambooDjangoRuntime.ConvergeMixin
`_set_node_status()` 手动状态管理	BambooDjangoRuntime.StateMixin + post_set_state 信号

节点状态追踪通过 [pipeline.eri.signals.post_set_state] 信号异步完成，[signals.py] 中的 [on_post_set_state] 处理器将 bamboo 状态映射到 FlowExecution 状态。

2. Executor Factory 模式

原子层通过 Factory Pattern 实现多平台支持：

```
meta.json (executor_type)
  → AtomExecutorFactory
    → importlib.import_module 惰性加载
    → BaseExecutor.execute()
    → ExecuteResult(success, data, error)
```

新增平台只需：写 [*_executor.py] 继承 [BaseExecutor] → 注册到 FACTORY 字典 → 创建 [meta.json] ([executor_type] 指向新类名)。

3. Component 注册机制

通过 [pipeline.component_framework.Component] 元类将原子注册到全局 ComponentLibrary：

```
ATOM_REGISTRY (meta.json 扫描)
  → register_atom_services()
    → type() 动态创建 Component 子类
      → ComponentMeta 自动注册到 ComponentLibrary + ComponentModel
        → BambooDjangoRuntime.get_service(code) 查询 Service
```

每个原子共享同一 [AnsibleAtomService] 基类执行逻辑，通过 [_atom_name] 区分类型。

4. AI 集成

- 使用 DeepSeek via OpenAI-compatible API
- [response_format={'type': 'json_object'}] 保证输出 JSON
- System Prompt 动态注入可用原子列表（从 AtomRegistry 读取，排除 [shell] 原子防止 AI 用作 fallback）
- SafetyGuard 后端校验，包含白名单/高危/备份检查
- 多重 AI 幻觉防御：
- [_errors] 字段检测：AI 无法完成请求时生成 [_errors]，后端拒绝保存
- Shell 原子过滤：从 AI 可见原子列表移除 [shell] + 服务端二次拦截
- 跨平台误用检测：用户输入含 VM/虚拟机 时拦截 AI 使用 netapp_* 原子
- RAG 搜索 OpsKnowledge 注入相关案例

5. 布局引擎

Sugiyama 分层图布局引擎（适配 bk_sops [pipeline_web/drawing_new]），提供确定性的节点自动布局，替代早期的 LLM 布局方案：

- 5 阶段算法：环移除 → 层级分配 → 交叉最小化 → 虚拟节点 → 坐标分配
- 无 LLM 依赖，纯 Python 实现，毫秒级完成
- 端点：[POST /api/opsflow/templates/ai_layout/]
- 支持所有节点类型（原子/网关/事件），自动合成起止节点